CSC 548 · PARALLEL SYSTEMS

Final Project

Topics & guidelines

Instructor: Jiajia Li

Fall 2026

09/09/2026

Agenda

1 · OVERVIEW

What the project is, solo or pairs, what to deliver.

2 · TOPICS

Suggested themes — accelerate, reproduce, or model.

3 · DELIVERABLES & TIMELINE

Proposal, presentation, report + code, due dates.

4 · TIPS

Pick something measurable, baseline first, profile, show scaling.

ONE THEME

A good project takes a **real workload**, parallelizes it in C/C++ with OpenMP / Pthreads / MPI / CUDA, and **quantifies** the win.

MODULE 1

Project Overview

What Is the Final Project?

> **THE CORE ASK**
> Take a real problem, make it run in **parallel**, optimize it, and **analyze** the performance you obtained.

It is open-ended on purpose: you choose the application and the parallel programming model. What stays fixed is the engineering discipline — a baseline, a parallel version, and an honest measurement of the difference.

> **THREE VALID SHAPES**
>
> **Accelerate** — speed up a problem with parallel programming.
>
> **Reproduce** — re-implement a parallel algorithm / high-performance result from a research paper.
>
> **Survey** — review a parallelism / performance topic. Not recommended.

Team Size & Expectations

SINGLE-PERSON

You scope, build, and analyze the whole project yourself.

TWO-PERSON TEAM

Allowed — at most **two** students. **More work is expected** from a team.

PLAN FOR THE TEAM FACTOR

If you pair up, the scope should be visibly larger — e.g. more optimizations, more platforms compared, or a deeper analysis. Split the work so each person owns a clear part.

Implementation Rules

LANGUAGE

Restricted to **C** or **C++**.

PARALLEL MODEL

No restriction — multithreading (OpenMP, Pthreads), GPU (CUDA, OpenCL), or distributed (MPI, Hadoop, Spark).

MACHINES

Not limited to ARC or Hazel — use any suitable machine (ARC, NCSU HPC / Hazel, a GPU box, ...). Mind the available resources.

PICK THE MODEL THAT FITS THE PROBLEM

Shared-memory loops → OpenMP / Pthreads.
Massively data-parallel kernels → CUDA.
Problem too big for one node → MPI.
Match the tool to where the parallelism actually lives.

MODULE 2

Suggested Topics

General Directions

ACCELERATE

Use parallel programming to speed up a problem you care about.

REPRODUCE

Re-implement a parallel algorithm / high-performance result from a published paper.

SURVEY

A literature survey on a parallelism / performance topic. **Not recommended.**

BEST BET

The "accelerate" and "reproduce" tracks give you real code to benchmark — which is what the report and grade reward. Choose one of those unless you have a strong reason not to.

Potential Topics

Theme	What you might do
Accelerating DL / ML models	Speed up **training** or **inference**; combine with pruning, sparsification, or quantization.
Accelerating data analytics	Analyze healthcare / social-media data with a method, then parallelize that method; or speed up data reorganizing, rearranging, cleaning.
Improving fundamental operations	Linear algebra, tensor algebra, matrix solvers — exploit data features like **sparsity**, **symmetry**, or **non-negativity**.

> **PATTERN**
> Each theme is "a real workload + a method to parallelize." Pick a workload, find the hot computation, and make that parallel.

Other Acceptable Topics

PERFORMANCE MODELING

Modeling a parallel algorithm's performance is fine on its own.

PROFILING / ANALYSIS

OK — but must come with **at least one optimization** driven by what you found.

BENCHMARKING

Acceptable — must compare **multiple** cases, scenarios, or platforms.

I/O OPTIMIZATION

I/O-focused optimization work is acceptable.

COMMON THREAD

Even the "analysis-flavored" topics must end in something concrete — an optimization, a comparison, or a model — not just observations.

Candidate Topics — Block-Sparse Tensor Contraction

The next five topics all build on TIDES, a just-accepted SC 2026 paper from the instructor's group on tiered block-sparse tensor contraction. Each takes the same kernel — block-sparse tensor contraction — to a different platform: multicore CPU, distributed nodes, AMD GPU, Intel GPU, and Tenstorrent.

WHY PICK ONE

These are open, active research directions — a solid project here could grow into a publication. Ask the instructor for more information to start from.

REFERENCE

Z. Li, S. H. R. Deverapalli, Y. Zhao, F. Mueller, K. Pierce, J. Li. "TIDES: Tiered Block-Sparse Tensor Contraction with Streaming Transpose on GPUs." The International Conference for High Performance Computing, Networking, Storage, and Analysis (SC), 2026 (accepted).

Code: **github.com/tensorworld/tides**
Datasets: **TensorSuite — quantum physics**

Candidate Topic — Block-Sparse Tensor Contraction on a Multicore CPU

Tensor contraction generalizes matrix multiply to many-dimensional tensors. When the tensors are block-sparse — whole blocks vanish by symmetry or structure (quantum chemistry, DMRG / quantum many-body, some GNNs) — the contraction becomes a large batch of small dense GEMMs over the surviving blocks.

NODE-LEVEL PARALLELISM

Parallelize across the cores of one CPU with OpenMP: spread the block-GEMMs over threads, vectorize (AVX, a tuned micro-kernel, or a BLAS call), and handle the load imbalance from uneven block sizes.

BUILD & MEASURE

- a correct serial block-sparse TC baseline;
- a threaded version — static vs dynamic scheduling of blocks;
- thread scaling (1 … all cores), GFLOP/s vs machine peak, a roofline (it is memory-bound);
- how block size and sparsity pattern affect balance.

Instructor's research area — ask for dataset and reference pointers.

Candidate Topic — Distributed Block-Sparse Tensor Contraction

The same block-sparse tensor contraction, but now too large for one node — distribute the tensors and the contraction across nodes with MPI. Each rank owns a subset of blocks and runs its local block-GEMMs; blocks must be exchanged so the contracted indices line up.

THE HARD PARTS

Partitioning blocks to balance work, moving only the blocks a rank actually needs, and overlapping that communication with local GEMMs — the sparse structure makes the communication pattern irregular.

BUILD & MEASURE

- a distributed block-sparse TC (start from a 1-D block distribution);
- messages and bytes per contraction — needs-only exchange vs broadcasting everything;
- strong and weak scaling across nodes;
- per-rank load imbalance vs the partitioning you chose.

A natural step up from HW1's distributed SpMV: same partition / communicate / overlap ideas, richer structure.

Candidate Topic — Block-Sparse Tensor Contraction on an AMD GPU

Port the block-sparse tensor contraction to an AMD GPU with HIP / ROCm. Each surviving block is a small dense GEMM, so the kernel is a large batch of small GEMMs — via rocBLAS batched/strided calls or a custom tile kernel.

GPU CHALLENGES

Small GEMMs underuse the GPU: watch occupancy, keep the batch on-device, balance blocks across work-groups, and remember it is memory-bound.

BUILD & MEASURE

- a HIP port of the batched block-GEMM;
- speedup vs the CPU baseline; achieved bandwidth vs an AMD-GPU roofline;
- profile with rocprof / Omniperf;
- effect of block size on occupancy and balance.

Candidate Topic — Block-Sparse Tensor Contraction on an Intel GPU

The same batched block-GEMM on an Intel GPU (Xe), written in SYCL / oneAPI (DPC++). Use oneMKL's batched GEMM, or a custom SYCL kernel, to run the surviving blocks.

GPU CHALLENGES

Same small-GEMM inefficiency; map blocks to sub-groups / work-items, keep data resident, and balance uneven blocks across the Xe cores.

BUILD & MEASURE

- a SYCL / oneAPI port (oneMKL batched GEMM as the baseline kernel);
- speedup vs CPU; achieved bandwidth vs an Intel-GPU roofline;
- profile with VTune / Advisor;
- compare portability and performance against the AMD / CUDA versions.

Candidate Topic — Block-Sparse Tensor Contraction on Tenstorrent

Map the block-sparse tensor contraction onto a Tenstorrent accelerator — a grid of Tensix cores over a NoC, programmed with tiles and circular buffers via TT-Metalium / TT-NN. Each block becomes one or more 32×32 tiles streamed through the compute engines.

ACCELERATOR CHALLENGES

Tile the blocks to the hardware tile size, place data and stream it over the NoC, and schedule the surviving blocks across the Tensix grid — the dataflow model is very different from a GPU.

BUILD & MEASURE

- a TT-Metalium implementation of the batched block-GEMM;
- Tensix-grid utilization and NoC data movement;
- throughput vs a CPU / GPU baseline;
- how block size maps to tiles and affects efficiency.

See the Topic 13 "Other GPUs & AI Accelerators" deck for Tenstorrent background.

MODULE 3

Deliverables & Timeline

The Three Deliverables

Deliverable	What it is	Due (Fall 2026)	Weight
Proposal	1 page — problem, plan, method, machine	Sep 16, 11:59pm ET	no grade
Presentation	4 min talk + 30 sec Q&A, in class	Nov 18 / 23 / 30	10%
Final Report + Code	4 pages min. (figs/tables incl., refs excl.) + code, ACM 2-column	Dec 7, 11:59pm ET	20%

WHERE TO SUBMIT

All artifacts — proposal, slides, final report + code — go through Moodle.

READ THE TIMELINE BACKWARDS

> The report needs results; results need a working parallel version; that needs a baseline. Start the baseline right after the proposal.

Writing the Final Report

FORMAT

4 pages minimum (including figures/tables, excluding references), ACM 2-column template — LaTeX (Overleaf) or Word. Submit the code alongside it.

A typical structure: problem & motivation → baseline & method → parallel design → experimental setup → results (speedup / scaling) → analysis & conclusion.

SHOW, DON'T CLAIM

Include the things a reader needs to believe you: • machine & thread/GPU/node counts

- baseline timing & how you measured
- speedup and a scaling plot
- what the profiler said you fixed

MODULE 4

Tips for a Good Project

Pick Something Measurable

> **DEFINE THE METRIC FIRST**
> Decide up front what "better" means — runtime, throughput, speedup, or scaling efficiency — and how you will measure it.

A project you can put a number on is a project you can defend. Avoid scopes where success is fuzzy ("it feels faster").

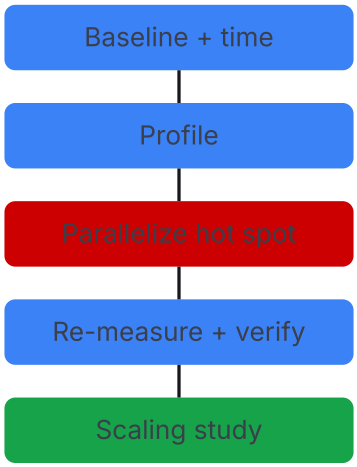
> **SCOPE REALISTICALLY**
> One application, one clear hot spot, on whatever machine fits. A small problem done thoroughly beats an ambitious one with no results.

Baseline First, Then Profile

WORKFLOW

1. Correct serial baseline + timing.
2. Profile to find the hot spot.
3. Parallelize / optimize that spot.
4. Re-measure; verify results still correct.
5. Sweep threads / cores / problem size.

Never optimize before you have measured. The profiler, not intuition, tells you where the time goes.



Show Speedup & Scaling

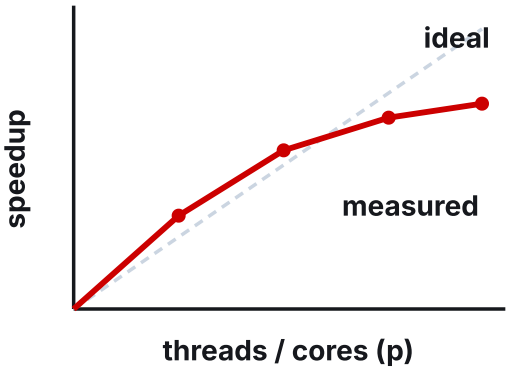
SPEEDUP

$S_p = T_1/T_p$ — serial time over parallel time on p workers.

STRONG VS. WEAK SCALING

Strong: fixed problem, more workers. **Weak**: grow the problem with the workers.

Amdahl's law caps strong scaling by the serial fraction — expect curves to bend, and say why.



Plot measured vs. ideal; explain the gap.

Common Pitfalls to Avoid

NO BASELINE

Speedup is meaningless without a correct, timed serial reference.

WRONG ANSWERS, FAST

Always verify the parallel output matches the baseline.

SINGLE DATA POINT

One thread count tells you nothing — sweep p and problem size.

OVER-SCOPING

Too ambitious → no results. Narrow it so you can finish and measure.

OPTIMIZING BLIND

Tuning before profiling wastes effort on cold code.

LATE START

Leaving experiments to the last week leaves no room to debug.

Final Project — Big Picture

CHOOSE

A real C/C++ workload to **accelerate** or a paper to **reproduce**.

BUILD

Baseline → profile → parallelize with OpenMP / CUDA / MPI on a suitable machine.

MEASURE

Speedup & scaling; verify correctness at every step.

DELIVER

1-pg proposal → 4-min talk + 30 sec Q&A (10%) → 4-pg ACM report + code (20%), all on Moodle.

ONE SENTENCE

Pick something measurable, baseline it, profile it, parallelize the hot spot, and report honest speedup & scaling.

References

> **COURSE MATERIAL**
> Adapted from the CSC 548 T3 — Final Project Topics handout; deliverables, format, and Fall 2026 dates follow the course page.

- ACM Primary Article Template (2-column) — **acm.org publications template**.
- NCSU HPC — ARC and campus HPC / Hazel documentation (OIT / HPC).

Speaker notes